OPEN SOURCE SOFTWARE · MIDTERM

오픈소스소프트웨어 중간고사 정리

강의자료 11개 + 실습 가이드 3개(5-2 · 6-1 · 7-1)를 한 권으로

범위: AI 시대 왜 오픈소스인가 · OSS 이해 · OSS 라이선스 · 클라우드/AWS · EC2 · 리눅스 소개/vi · 패키지 관리 · 리눅스 명령어 · Git 기초 · Git 원격 저장소

★ 시험에 나오기 좋은 핵심 ■ 핵심 정리 ■ 헷갈리는 함정 ■ 실습 가이드 내용

목차

00	AI 시대, 왜 오픈소스인가
01	OSS 이해하기 — 용어, 정의, 원칙, 장단점, 역사, 참여
02	OSS 라이선스 — 스펙트럼, GPL/LGPL/MPL/BSD/Apache, 양립성, 이중 라이선스
03	클라우드 & AWS 계정 — 개념, 가입, Free Plan, 루트/IAM/MFA
04	AWS EC2 — Region/AZ, Budgets, Key Pair, 인스턴스 생성, 보안 그룹
05	리눅스 소개 & vi 에디터
06	패키지 관리 — rpm, dnf, httpd 설치 실습
07	리눅스 명령어 + 5-2 실습 가이드
08	Git 기초 + 6-1 로컬 저장소 실습
09	Git 원격 저장소 + 7-1 원격·협업 실습
10	헷갈리는 개념 비교 모음
11	연도 · 숫자 암기표
12	예상 문제 (정답·해설은 맨 뒤)

시험 직전에 이것만은

1. 라이선스 비교표: **GPL만** 독점 SW와 결합 불가 · BSD/Apache는 2차 저작물 공개 의무 X
2. GPL v2 → v3: Tivoization 금지, 특허 조항, Apache 2.0과 양립 가능
3. EC2 **중지(Stop)**는 EBS 비용 계속 / **종료(Terminate)**는 완전 삭제·되돌릴 수 없음
4. 심볼릭 링크(경로 참조, 원본 삭제 시 깨짐) vs 하드 링크(inode 참조, 원본 지워도 유지)
5. `chmod` 숫자 계산(r=4, w=2, x=1), `>` / `2>` / `<` / `|`
6. Git 4공간: Working Dir → (add) → Staging → (commit) → Local Repo → (push) → Remote
7. `git diff` / `--cached` / `HEAD` 의 비교 대상
8. 태그는 `git push` 만으로 안 올라감 → `git push origin v1.0.0` 또는 `--tags`

00 AI 시대, 왜 오픈소스인가

현실 인식 — 신입 채용 시장

지표	값	의미
SW 인력 증가율	**0.2%**	2024년 7.3% → 2025년 0.2%, 고용 확대가 사실상 멈춤
인력 충원 계획 기업	**24%**	신규 채용 계획이 있는 기업은 4곳 중 1곳
경력직 : 신입 채용	**2.8배**	경력직 1만 5천 명 vs 신입 5,300명

"월 10달러짜리 코파일럿이 연봉 9만 달러 신입보다 효율적" → 단순 코딩의 시대는 끝났다. 기업이 보는 능력은 **창의성 · 문제해결 · 협업**(이번 학기 전체를 관통하는 키워드).

AI 인프라의 뼈대는 오픈소스

프로젝트	설명	프로젝트	설명
PyTorch	메타(Meta)가 공개한 딥러닝 프레임워크	Linux	AI 서버의 표준 OS
TensorFlow	구글이 공개한 머신러닝 프레임워크	Kubernetes	구글발, AI 서비스 배포·운영 표준
Hugging Face	AI 모델·데이터셋 공유 생태계	Python	AI 개발의 사실상 표준 언어

Git · Linux · Docker는 이 생태계의 **"공용어"**.

> **오픈 웨이트(open-weight)**
> AI 모델을 학습시킨 결과물인 **가중치(weight) 파일 자체를 공개**해 누구나 내려받아 실행하거나 다시 학습·수정할 수 있게 한 것. 예: Meta **Llama**, **DeepSeek**(낮은 비용·높은 성능), Alibaba **Qwen**(다국어), Moonshot AI **Kimi K3**(2026년 7월, 2.8조 파라미터).

OSS 기록 = 가장 강력한 포트폴리오

이력서의 주장	GitHub이 보여주는 증거
Git·GitHub 활용 가능	실제 커밋 · PR 기록
협업 능력 우수	Maintainer와 주고받은 코드 리뷰
문제해결 능력 보유	버그를 고친 코드와 테스트 결과

몸에 익히는 4가지: **실전 경험 · 협업 역량 · 문서화·테스트 · 최신 기술 이해**. 사례: 2025년 한성대 학생 수상(멋쟁이사자처럼 해커톤 대상 *Park Here*, SW중심대학 디지털 경진대회 최우수상 *상상부기*).

이번 학기 로드맵

1단계 · 기초 다지기	2단계 · 협업 도구	3단계 · 실전 응용
1 OSS 개요·라이선스 2 리눅스 개요 3 클라우드·AWS	4 Git·GitHub 5 GitHub Actions 6 Docker	7 AI 코딩 도구 활용 8 OSS 기여 프로젝트

01 OSS 이해하기

용어 정리 ★

구분	주체 · 관점	핵심
Free Software	FSF · **윤리적** 관점	사용자의 **4대 자유**(실행 · 연구(수정) · 배포 · 수정본 재배포) 보장. GPL 등 카피레프트는 파생물도 같은 자유를 유지하도록 요구
Open Source Software	OSI · **실용적** 관점	소스 공개가 더 나은 SW를 만드는 방법. 공개 코드의 검증과 협업으로 품질·개발 속도 향상
Proprietary SW	Closed Source	소스 비공개, 저작권자가 허락한 범위에서만 사용·수정·배포

FOSS(Free Open Source Software), **FLOSS**(Free/Libre Open Source Software): 두 개념을 아우르는 표현. 국내에서는 "**공개 소프트웨어**".

OSS의 정의

> **정의**
> 저작권자가 소스코드를 공개하여, **누구나 자유롭게 사용·수정·재배포**할 수 있는 SW → 공유와 협업을 통한 지속적 발전

- 대표 라이선스: **MIT**(가장 느슨, 표시만 하면 자유) · **Apache 2.0**(MIT 유사 + 특허 조항) · **GPL**(카피레프트, 파생물도 동일 라이선스로 공개)
- OSS에도 **저작권은 존재**. 라이선스 의무(출처 표기, 재배포 조건 등)는 반드시 지켜야 함
- OSI에는 약 **80개**의 인증 라이선스가 등록

> **함정**
> 무료로 배포돼도 소스가 비공개인 프로그램(예: **카카오톡**)은 OSS가 **아니다** — 소스 공개도, 협업도 안 되니까. "무료 = 오픈소스"가 아님.

OSI 라이선스 핵심 원칙 5가지 ★

#	원칙	내용
1	자유 재배포	로열티·수수료 없이 누구나 재배포 가능
2	소스코드 공개	소스코드를 함께 공개
3	파생 저작물 허용	수정을 통한 2차 저작물 창작 허용 (라이선스에 따라 동일 조건 배포 요구)
4	무차별 원칙	**특정 개인 · 단체 · 사용 분야 · 기술**을 차별하면 안 됨
5	라이선스 독립성	특정 제품에 종속되지 않고, 함께 배포되는 다른 SW를 제한하지 않음

OSI의 전체 기준은 10개이며, 강의는 실무에서 가장 중요한 5개만 정리. "비상업적 용도로만 사용 가능" 같은 조건은 사용 분야 차별 → OSI 기준 OSS가 아님.

OSS의 두 가지 형태

① **패키지 형태** — 라이브러리·모듈 단위로 배포, 저작권·라이선스 고지문 포함. "**부품 단위**"(라이브러리 하나)

② **프로젝트 형태** — 완성된 시스템 전체가 소스 단위로 공개. "**완제품 단위**"(리눅스 OS 전체)

대표 오픈소스 스택: OS(Linux) · 웹 서버(Nginx, Apache) · DBMS(MySQL, PostgreSQL) · 언어(Python, JavaScript) · 프레임워크(Django, Spring, React) · 컨테이너(Docker, Kubernetes)

장점과 단점

장점 4가지

- **종속성 탈피**: 벤더 Lock-in에서 벗어남, 직접 버그 수정
- **비용 절감**: 초기 도입·유지 비용 감소
- **생산성·개발기간 단축**: 바퀴를 다시 발명하지 않음, 커뮤니티가 신뢰성·보안성 향상
- **개발 역량 증대**: 알고리즘·설계를 직접 분석·응용

단점과 대응

단점	대응
품질 보증 부족 (보증 기업 없음, 지속성 불확실)	성숙한 커뮤니티·유지관리 이력 있는 프로젝트 선택
IPR 관련 잠재 위협 (영업비밀 유출, 라이선스 분쟁, 특허 침해)	체계적 관리 방안 (**Governance**) 도입

OSS 역사 연표 ★

연도	사건	연도	사건
1983	GNU 프로젝트 시작 (스톨만, Free Software 개념)	**1985**	FSF 설립
1989	GNU GPL v1 (최초의 카피레프트)	**1991**	Linux 커널 공개 (토발즈)
1998	OSI 설립 (에릭 레이먼드, "Open Source" 용어)	**1999**	Apache 재단(ASF) 설립
2005	Git 공개 (토발즈, 분산 VCS)	**2007**	GPL v3 발표
2008	GitHub 창립 (PR 기반 협업 전환점)	**2014**	Docker 공개 (컨테이너 대중화)
2015~	AI 프레임워크 공개 (TensorFlow, PyTorch)	**2023~**	오픈 웨이트 LLM 경쟁 (Llama, DeepSeek, Qwen)

OSS의 중요성

기업: 생산성 증대·비용 절감, No Vendor Lock-in, OSS 기반 비즈니스 모델(예: **Red Hat** = 오픈소스 + 유료지원/컨설팅)

개발자: 코딩 능력 제고, 협업·소통 능력, 개발 프로세스 전체(설계·개발·테스트·배포·모니터링) 이해

생태계 참여

Contributor → Committer 흐름: 찾아보기 → 살펴보기 → 문제 발견·제안(Issue) → 기여(PR) → 리뷰·병합(Maintainer). 꾸준히 기여해 신뢰를 쌓으면 **Committer**(코드 리뷰·병합 권한을 가진 핵심 관리자)로 성장.

- **순환 구조**: Core 팀(개발·문서화·테스트, 설계 변경 등 의사결정) ↔ 커뮤니티(버그 리포트·기능 요청(Issue), PR). 배포 단계: 최신 스냅샷 → 베타 → 안정 릴리즈. 커뮤니티 산출물: Patch, Bug Report, Forum 논의, Feedback
- **ALM**(Application Lifecycle Management): 요구사항부터 배포까지 SW 전체 생명주기 관리 체계. 이슈 등록 → 로컬 개발 → PR 제출 → 코드 리뷰·CI → 품질 검사(정적 분석·보안 스캔) → 병합·배포(CI/CD). GitHub Issues, PR 리뷰, **GitHub Actions**가 담당

02 OSS 라이선스

강의 2

저작권 · 특허 · 라이선스

구분	내용
저작권 (Copyright)	창작과 **동시에 자동 발생**. 타인이 쓰려면 저작자의 허락 필요
특허 (Patent)	발명에 대해 **별도로 출원**해 인정받는 독점권
라이선스 (License)	저작권자(Licensor)가 사용자(Licensee)에게 내리는 **사용 허가**. 조건 아래 사용·수정·배포 허용

라이선스가 명시되지 않은 프로젝트를 사용하는 것은 위험(법적 문제 가능).

현황과 AI가 불러온 리스크

- 전 세계 약 **2,000여 종** 라이선스. Permissive 비중 **78%**(2024), MIT 포함 상용 코드베이스 **92%**, Copyleft(GPL 계열) **22%** (과거 40%대에서 하락)
- MIT가 1위, Apache 2.0은 Kubernetes 등 클라우드 네이티브에서 강세, GPL은 Linux 커널 등 시스템 SW에 집중
- **라이선스 세탁(License Laundering)**: AI 코딩 도구가 GPL 코드 포함 저장소로 학습 → 생성 코드에 출처·라이선스 정보가 남지 않아 모르게 카피레프트 코드를 끌어옴. 2026년 상용 코드베이스 라이선스 충돌 발견율 **68%**, AI 코드의 IP 리스크 검토 기업 **54%**, 한 코드베이스 최대 **2,675건** 충돌

공통 준수사항 3가지 ★

준수사항	내용
① 저작권 문구 유지	소스의 저작권 문구를 **그대로 유지**. 수정했다면 기존 라이선스 아래에 **수정 사실·일자·수정자**의 저작권 문구를 **추가**
② 상표권 보호	OSS 제품명은 상표권으로 보호. 같은 이름을 내 제품명으로 쓰면 침해 가능
③ 라이선스 양립성 확인	서로 다른 라이선스 코드 결합 시 요구사항 충돌 여부 확인. 충돌하면 배포 자체가 불가능해질 수 있음

라이선스에 따라 달라지는 의무

- **사용 여부 명시**: "본 SW는 오픈소스 OO를 사용하였습니다"를 매뉴얼 등에 고지 (GPL·LGPL·MPL은 소스 입수 경로까지 구체적으로 고지 / MIT·BSD·Apache는 저작권 문구 유지로 충분한 경우가 많음)
- **소스코드 공개**: 수정·추가 부분 공개 (공개 범위는 라이선스마다 다름)
- **특허 조항**: **Patent Grant**(저작권자의 특허권도 무상 제공) / **Patent Retaliation**(사용자가 특허소송을 걸면 라이선스 자동 종료, 보복 조항)

라이선스 스펙트럼 ★

구분	별칭	라이선스	의무
Permissive	Give Me Credit	BSD, MIT, Apache	수정해도 소스 공개 의무 없음. 저작권 표시만 유지
Weak Copyleft	Give Me Fixes	LGPL, MPL, EPL	**수정한 부분만** 공개. 연결·결합된 나머지는 자유
Strong Copyleft	Give Me Everything	GPL, Affero GPL	**결합된 전체 프로그램**을 동일 라이선스로 공개

Copyright vs Copyleft

Copyright는 저작물을 보호하고 허락을 받게 하는 개념. **Copyleft**는 자유로운 사용·배포를 허용하되 **변경된 소스도 동일 라이선스로 공개**하게 해 파생물이 계속 자유롭게 유지되도록 함.

GPL — 가장 강력한 Copyleft (FSF)

GPL v2.0

- 수정하거나 라이브러리를 링크(**정적·동적 모두**)하면 SW 전체를 GPL로 공개
- **내부적으로만 사용(배포 X)하면 공개 의무 없음**
- 기업은 영업비밀까지 공개해야 해 사업 전략과 충돌 가능

GPL v3.0 — v2 대비 추가 ★

- **Tivoization 금지**: 소스만 주는 게 아니라 사용자가 고친 프로그램을 실제 기기에 설치·실행할 수 있게 해야 함
- **특허 조항** 추가 (Patent Grant, Retaliation)
- **호환성 조항**: Apache 2.0과 양립 가능

사례: Linux Kernel, MariaDB, WordPress. **Linux 커널은 지금도 GPLv2만** 사용 — 토발즈가 Tivoization 금지 조항에 반대.

LGPL · MPL — 절충형 Copyleft

LGPL 2.1 (Lesser GPL, FSF)

- FSF가 공개SW 확산을 위해 전략적으로 만든 라이선스
- **동적 링크**: 실행 시 불러오면 응용프로그램 소스 공개 의무 없음
- **정적 링크**: 컴파일해 넣으면 재링크할 수 있도록 오브젝트 파일 등 제공
- 기업은 GPL보다 LGPL 선호

MPL (Mozilla Public License)

- Netscape 브라우저 소스 공개를 위해 개발
- 수정한 코드를 **파일 단위**로만 공개, MPL로 배포
- 다른 라이선스 코드와 결합 가능(결합 부분은 다른 라이선스 허용)
- Patent Grant · Retaliation 포함

BSD · Apache — 대표 Permissive

BSD License

- 소스 공개 의무 없음(미국 정부 재원으로 개발)
- BSD 기반으로 개발 후 **소스 비공개로 판매 가능**

Apache License 2.0 (ASF)

- ASF의 모든 SW에 적용. 'Apache' **상표권** 침해 금지
- **Patent Grant**: 기여자의 특허권을 무상 실시 허락
- **Patent Retaliation**: 특허 소송을 걸면 특허 라이선스 자동 종료
- MIT와 유사한 권리, 상용 SW에서 제약 없이 사용
- **GPL 2.0과 비양립, GPL 3.0과 양립**

라이선스 한눈에 비교 ★★

구분	무료이용	소스 취득	소스 수정	배포 허용	2차 저작물 공개	독점SW와 결합	특허 조항
GNU GPL	O	O	O	O	O	**X**	△
GNU LGPL	O	O	O	O	O	O	△
MPL	O	O	O	O	O	O	O
BSD	O	O	O	O	**X**	O	**X**
Apache	O	O	O	O	**X**	O	O

△ = 버전에 따라 다름: GPLv2·LGPLv2.1은 명시적 특허 조항 없음, GPLv3·LGPLv3부터 추가. 2차 저작물 공개 의무가 있는 **GPL만** 독점(폐쇄형) SW와 결합 불가.

양립성 (Compatibility) ★

MPL(A) + GPL(B) 결합 예시

GPL은 결합된 **전체**를 GPL로 공개하라고 요구 → A도 GPL로 공개해야 하는데, A는 MPL로만 공개하기로 허락된 코드라 GPL로 재배포할 권한이 없음 → **'A+B'는 배포 자체가 불가능**. 한쪽을 지키면 반드시 다른 쪽을 어기는 구조 = 양립 불가능.

- Apache 2.0 ↔ GPLv3 : **양립 가능** (GPLv3의 호환성 조항)
- MPL 1.1 ↔ GPLv3 : **양립 불가능** (단, 2012년 MPL 2.0이 GPL·LGPL 결합 재배포를 명시적으로 허용)

- Permissive(Apache, BSD, MIT)는 일반적으로 양립성 문제를 크게 고려하지 않아도 됨

이중(Dual) 라이선스

형식	내용
'GPL or MIT'	상용 SW 개발 → MIT 선택 / 커뮤니티 기여 → GPL 선택
'GPL or Commercial'	**Commercial 라이선스를 구입하면 GPL 의무 없이** 상용 SW에 포함 가능. 사례: **MySQL**(GPL 2.0 배포, 상용 포함 시 상용 라이선스 구입)

실무 관리 (Governance) & FAQ

절차: 사용할 OSS 판단 → 라이선스 양립성 검토 → 사용 목록 작성·저장 → 소스 공개·사용 여부 고지. **외주업체 코드에도 동일 적용**. 위반 모니터링: FSF, SFLC, KOSS Law Center → 시정 요구 → 거부 시 소송.

Q	A
그냥 쓰기만 해도 의무가 생기나?	**배포하지 않으면** 대부분 의무 없음. 수정 후 **배포할 때** 소스 공개 의무 발생
상용 SW에 유리한 라이선스는?	소스 공개 의무 없는 **BSD·MIT** 등 Permissive
기업 도입 전 먼저 확인할 것은?	2차 저작물 공개 의무가 있는 라이선스(GPL, LGPL, AGPL, EPL)인지. 대체 불가면 공개 감수 또는 상용 라이선스 구매
'소스 공개'의 범위는?	GPL은 **링크된 코드까지**, LGPL은 **링크된 코드 제외**

03 클라우드 & AWS 계정

강의 3-1

클라우드 컴퓨팅 정의

정의
인터넷을 통해 컴퓨팅 파워, DB, 스토리지, 애플리케이션 등 IT 자원을 **필요한 만큼 즉시 빌려 쓰고, 쓴 만큼만 지불**하는 방식.
비유: 직접 발전소를 짓지 않고 전력회사에서 전기를 쓴 만큼 요금을 내는 것.

특징	내용
On-Demand	필요할 때 즉시, 클릭 몇 번으로 자원 확보
Pay-as-you-go	쓴 만큼만 지불, 초기 투자 비용 없음
Self-Service	IT 부서 요청 없이 웹 콘솔에서 직접 구성

On-Premise vs Cloud

On-Premise (발전소를 직접 지음)	Cloud (전기를 쓴 만큼 냄)
서버·인프라 직접 구축, 초기 비용 큼	필요할 때 바로 연결, 구축 불필요
사용량과 무관한 고정 비용	사용한 만큼 요금(종량제)
유지보수·운영 직접 담당	유지보수는 클라우드 사업자 담당

클라우드의 4가지 이점: 비용 효율(Cost Efficient) · 빠른 글로벌 배포(Deploy in Minutes) · 확장성(Scalability) · 신뢰성·가용성(Fault Resilient)

시장 현황

점유율: **AWS 28%** · Microsoft Azure 21% · Google Cloud 14% (세 회사 합계 약 63%, 2026 Q1). AWS는 **2006년** 가장 먼저 시작한 선발주자, 200개 이상 서비스. 성장률(2026 Q2): Azure +43%, Google +82%, AWS +37%.

AWS 계정 생성 8단계

① 이메일·계정명 → ② 이메일 인증 → ③ 비밀번호(루트 사용자) → ④ 계정 플랜(무료 6개월/유료) → ⑤ 연락처·주소 → ⑥ 결제 정보(카드, **1 USD/100원** 3~5일 결제 대기 후 자동 취소) → ⑦ 전화(SMS) 인증 → ⑧ Support 플랜(학습용은 무료 '기본 지원')

Free Plan vs Paid Plan

Free Plan	Paid Plan
가입 즉시 **$100** 크레딧 + 미션 완료 시 최대 **$100** 추가 최대 **6개월** 또는 크레딧 소진 시 만료(**계정 자동 종료**) 신규 고객 전용	동일 크레딧, 모든 서비스 제한 없이 사용 소진 후 **pay-as-you-go로 자동 전환**(계정 유지)

EC2 1년 무제한 무료 시대는 끝남 → 크레딧 + 미션 방식으로 변경.

계정 보안 — 루트 vs IAM ★

① 루트 계정 MFA 설정

- 루트 = 결제 정보 변경·계정 삭제 등 **모든 권한**(만능 열쇠)
- 비밀번호 + 스마트폰 앱(OTP, Google Authenticator)/보안 키로 추가 인증
- 루트가 뚫리면 계정 전체와 결제 수단까지 탈취

② IAM 관리자 사용자 생성

- IAM = Identity and Access Management
- 콘솔 → IAM → Users → Add users → 'admins' 그룹 + **AdministratorAccess** 정책
- 이후 모든 작업은 IAM 사용자로, 루트는 결제 변경 등 꼭 필요할 때만

순서: 루트 로그인 → MFA 설정 → IAM 관리자 생성 → 이후 IAM 사용자로 로그인. AWS 공식 모범 사례 1순위 = "루트 사용자 일상적 사용 금지".

04 AWS EC2

강의 3-2

Region과 Availability Zone

Region	Availability Zone (AZ)
전 세계에 분산된 **독립적인 데이터센터 클러스터**, 지리적 단위. 예: 서울 `ap-northeast-2`	한 Region 안의 **물리적으로 분리된 복수의 데이터센터**. 서울은 4개: `2a, 2b, 2c, 2d`

서버 4대 배치	특징
4개 AZ에 1대씩	**가용성 최대** — 한 AZ 장애에도 나머지 3곳 유지
2개 AZ에 2대씩	가용성과 비용의 균형, **실무에서 가장 흔한 절충안**
1개 AZ에 4대	가장 저렴·단순, 그 AZ 장애 시 **전체 중단**

가용성(Availability) ↔ 네트워크 비용·복잡성의 Trade-off.

AWS Budgets

- **제로 지출 예산**: $0.01 초과 시 즉시 알림 → 실습 계정에서 가장 먼저 설정 권장
- 월별 비용 예산 / 예약 사용량 예산(사용률이 목표 아래로 떨어지면 알림)

Key Pair와 SSH 인증 ★

- EC2는 username/password 대신 **공개키 암호(public-key cryptography)**로 인증. SSH는 **TCP 22번 포트**
- Key Pair = Public Key + Private Key. 콘솔에서 생성 → Private Key(`*.pem`) 다운로드(다시 받을 수 없음), Public Key는 EC2의 `~/.ssh`에 보관
- 경로: EC2 콘솔 → 네트워크 및 보안 → 키 페어. IAM 사용자도 `ec2:CreateKeyPair` 권한만 있으면 생성 가능

> **인증 원리 (Alice ↔ EC2)**
> ① "I'm Alice" → ② EC2가 임의의 난수 R 전송 → ③ Alice가 **개인키로 R을 암호화**해 [R]Alice 전송 → ④ EC2가 **Alice의 공개키로 복호화**해 R′ 획득, **R = R′이면 인증 성공**

EC2 인스턴스

EC2 Instance = AWS가 물리 서버 위에 하이퍼바이저로 만들어준 가상 머신(VM). 호스트에 하이퍼바이저 설치 → 여러 VM 실행 → 하드웨어 자원 공유 → 여러 OS를 한 물리 서버에서 실행.

생성 단계	내용
이름·태그	인스턴스 이름
AMI	Amazon Machine Image = **OS 설치 이미지 + 기본 세팅값** (예: Amazon Linux 2023)
인스턴스 유형	하드웨어 사양(스펙) 규격. 예: **t3.micro** = 범용, vCPU 2, RAM 1GiB, 시간당 $0.013(월 약 $9.4)
키 페어	SSH 접속용
네트워크 설정	보안 그룹 규칙: SSH(22), 필요 시 HTTP(80)/HTTPS
스토리지	루트 EBS 볼륨, **gp3**(가장 기본적인 SSD 타입)

Security Group & VPC

- **보안 그룹**: 인스턴스로 들어오는 트래픽을 포트·소스 단위로 허용하는 가상 방화벽 (예: 80 허용)
- 구조: **Region → VPC → Subnet → Instance**. 한 Region에 여러 VPC, VPC 안에 여러 Subnet. Subnet들은 여러 AZ에 분산되지만 **Subnet 1개는 AZ 1개에만** 속함

중지 vs 종료 ★★

중지 (Stop)	종료 (Terminate)
컴퓨팅 비용은 멈추지만 **EBS 볼륨(스토리지) 비용은 계속** 발생 나중에 다시 시작 가능 (퍼블릭 IP는 바뀜)	인스턴스와 볼륨 삭제, 비용 **완전히 중단** **되돌릴 수 없음**(재사용 불가)

접속 방법

① 웹 브라우저(**EC2 Instance Connect**) — 설치 없이 즉시 ② **SSH 클라이언트** — `.pem`으로 터미널 접속. 접속 전 확인: 인스턴스 상태(실행 중) · 퍼블릭 IPv4 주소.

```
$ chmod 400 my-key.pem                # 소유자만 읽기 (필수)
$ ssh -i my-key.pem ec2-user@<퍼블릭 IP>
```

Windows 명령 프롬프트에는 chmod가 없음 → Git Bash, WSL, 또는 PowerShell의 `icacls`로 권한 제한.

05 리눅스 소개 & vi 에디터

강의 4-1

What is Linux?

- **Free Unix 계열 운영체제. Linus Torvalds**가 처음 만들고 전 세계 개발자의 협업으로 발전
- 리눅스 커널은 **GPLv2**로 배포, 소스 공개 → 누구나 사용·수정·배포 → 다양한 배포판 탄생

배포판의 계층 구조

계층	역할 · 예
Application	Window Manager(GNOME, KDE), Apache Web Server, Mail Server 등
Shell	**명령 해석기** — bash, csh 등
Kernel	메모리/프로세스 관리, 하드웨어 관리(CPU, Memory, HDD…)

모든 배포판이 이 구조를 공유하지만 Shell 종류·응용 프로그램·설치 도구 등 세부 구성은 다름.

대표 배포판

계열	배포판	특징
Red Hat	**Fedora**	커뮤니티·오픈소스, 최신 기능 (최신 44)
	RHEL	상용(Commercial). Fedora 기반 + 장기 지원(LTS) 안정 플랫폼 (최신 10.2)
	CentOS · Rocky · AlmaLinux	RHEL 소스에서 Red Hat 상표만 제거한 **커뮤니티 재배포판** (Red Hat 제품 아님)
Debian	**Ubuntu**	Debian 기반, 쉬운 데스크톱, 초보자 추천
SUSE	openSUSE	SUSE가 후원하는 커뮤니티 배포판

CentOS 변화

전통적 CentOS Linux는 **2024년 6월(CentOS 7 EOL)** 종료. 지금은 RHEL의 업스트림 개발판인 **CentOS Stream**만 남음. 무료 RHEL 호환 서버 역할은 **Rocky Linux · AlmaLinux**가 이어받음.

	RHEL 계열	Ubuntu
초보자	자료가 서버·엔터프라이즈 위주	**추천** — 커뮤니티 큼, 데스크톱 인기
기업	**추천** — 느린 업데이트, 검증된 안정 버전, 안정성·보안 → 웹/DB 서버	최신 SW, 데스크톱·개인용에 적합
패키지	`.rpm` + yum (최신 **dnf**)	`.deb` + apt-get (최신 **apt**)

RHEL 8 이상·최신 Fedora는 dnf가 기본이며, yum 명령은 dnf의 별칭으로 동작.

vi 에디터 — 3가지 모드 ★

모드	진입	하는 일
Command	vi 시작 시 **기본**, Esc로 복귀	커서 이동(`h j k l`), `dd` 줄 삭제, `yy` 줄 복사, `p` 붙여넣기, `x` 글자 삭제
Insert	`i` , `a` , `o`	텍스트 입력
Ex (Last line)	`:`	저장·종료, 검색·치환

저장 · 종료	
`:w`	저장
`:q`	종료
`:wq`	저장 후 종료
`:q!`	저장하지 않고 종료

검색 · 치환	
`/text` · `?text`	아래(앞) · 위(뒤)로 검색
`:s/old/new/`	현재 줄, 첫 번째만
`:s/old/new/g`	현재 줄 전체
`:%s/old/new/g`	**파일 전체**

치환 외우는 법

끝의 **g** = 그 줄의 모든 것(없으면 첫 번째만) · 앞의 **%** = 파일 전체 줄(없으면 현재 줄만)

실습 팁 (5-2)

파일 끝에 줄 추가: `G` (맨 끝으로) → `o` (새 줄 + 입력 모드) → 입력 → `Esc` → `:wq` . 실수하면 `Esc` 후 `u` (undo) 또는 `:q!` .

06 패키지 관리 — rpm · dnf · httpd

강의 4-2 · 4-3

패키지와 관리 도구

패키지 = 하나의 SW 애플리케이션을 구성하는 **파일과 메타데이터의 묶음**. 배포판에 맞는 형식으로 포장 (예: httpd, mysql-server, openssh-server, git, python).

계열	포맷	관리 도구
Red Hat (RHEL, Fedora, CentOS)	`.rpm`	rpm → yum → **dnf**
Debian (Debian, Ubuntu)	`.deb`	dpkg → apt / apt-get

RPM (Red Hat Package Manager)

- 메타데이터: 버전, 포함 파일 목록, 설명, 패키저, **의존성** 정보
- 배포자가 **GPG 키로 전자 서명** → 무결성·출처 보장

옵션	의미	옵션	의미
`-i`	install	`-U`	upgrade
`-e`	erase(삭제)	`-q`	query(조회)
`-v`	verbose(상세)	`-h`	hash(# 진행률)

```
$ rpm -ivh mypkg-2.7-5.x86_64.rpm   # 설치
$ rpm -Uvh mypkg-2.7-5.x86_64.rpm   # 업그레이드
$ rpm -e mypkg                      # 삭제
$ rpm -q mypkg                      # 설치 여부 조회
```

파일명 규칙 ★

`mypkg-2.7-5.x86_64.rpm` → 이름(mypkg) · **버전**(2.7) · 릴리스(5) · 아키텍처(x86_64). 예: `httpd-2.4.68-1.amzn2023.0.1.x86_64`의 버전은 **2.4.68**.

RPM의 단점

매우 원시적(저수준 설치/삭제만). **의존성을 사용자가 직접 해결**해야 함 → `error: Failed dependencies`. 이를 해결하려고 **YUM**, 오늘날엔 후속인 **DNF** 등장.

DNF (Dandified YUM)

- RPM 기반으로 더 쉽고 편한 관리. 공식·서드파티 **저장소(repository)**에서 설치·삭제·조회
- **의존성을 자동으로 해결**
- 저장소 정보: `/etc/yum.repos.d/` (하위 호환 때문에 yum 시절 경로 그대로). 파일마다 `.repo` 저장소 하나

흐름(httpd 예): `dnf install httpd` → 저장소 메타데이터 요청 → 의존성 확인(apr, apr-util 필요) → httpd + apr + apr-util 다운로드 → rpm으로 3개 설치.

```
[amazonlinux]                              # /etc/yum.repos.d/amazonlinux.repo
name=Amazon Linux 2023 repository          # 설명
mirrorlist=https://cdn.amazonlinux.com/... # 서버 주소
gpgcheck=1                                 # 서명 검증 여부
enabled=1                                  # 사용 여부(1/0)
```

dnf 명령 정리 ★

명령	기능
`dnf check-update`	업데이트 가능한 패키지 확인
`sudo dnf update`	설치된 **모든** 패키지 업데이트 (관리자 권한 필요)
`sudo dnf update httpd`	지정한 패키지 **하나만** 업데이트
`dnf info httpd`	상세 정보 (설치됨 → 로컬 정보 / 미설치 → 저장소 정보)
`dnf list installed` / `available`	설치된 / 설치 가능한 패키지 목록
`dnf list nginx`, `dnf list mysql*`	이름(패턴)과 일치하는 패키지 조회 (설치 여부 무관)
`dnf search nginx`	키워드로 이름 + **요약(summary)** 검색
`sudo dnf -y install pkg`	설치 (`-y` : 모든 질문에 yes). 로컬 .rpm 경로도 가능
`sudo dnf remove pkg`	패키지와 **그 패키지에 의존하는 패키지들까지** 제거

> **search vs list**
> **search**는 이름과 요약에서 키워드 검색(정확한 이름을 모를 때). **list**는 이름(패턴)으로만 조회, 설명 텍스트는 검색하지 않음.

Apache 웹 서버 설치 실습 (4-3)

HTTP: 웹 클라이언트(브라우저)가 **HTTP request** → 웹 서버가 **HTTP response**. TCP/IP 위에서 전송. 웹 서버는 요청 수신 → 문서 탐색 → 응답 전송. **Apache**는 오픈소스·크로스 플랫폼 대표 웹 서버.

```
$ dnf list installed | grep httpd   # 1. 설치 여부 확인
$ dnf info httpd                    # 2. 정보 확인
$ sudo dnf install httpd            # 3. 설치
$ sudo systemctl start httpd        # 4. 서비스 시작
$ sudo systemctl status httpd       #    상태 확인
$ sudo systemctl enable httpd       #    부팅 시 자동 시작
$ sudo dnf remove httpd             # (필요 시) 제거
```

- EC2 보안 그룹 인바운드에 **TCP 80번**을 허용해야 외부 브라우저에서 접속 가능
- 브라우저에 퍼블릭 IP/DNS 입력 → Apache Test Page가 보이면 성공

07 리눅스 명령어 + 5-2 실습

강의 5-1 · 실습 5-2

명령어 구조 · 도움말 · sudo

`ls -l /etc` → **명령어 이름**(ls) · **옵션**(-l, 동작 방식 변경) · **인자**(/etc, 작업 대상)

- `man ls` : 매뉴얼. `f` /Space 다음 페이지, `b` 이전, `/검색어`, `q` 종료. man이 없으면 `ls --help`
- `sudo` : 관리자 권한으로 실행. EC2의 ec2-user는 **root 비밀번호가 없고** sudo로만 권한 사용. `sudo -i` = root로 전환

표준 입출력과 리다이렉션 ★★

스트림	번호	기본값
stdin (표준 입력)	0	키보드
stdout (표준 출력)	1	화면
stderr (표준 에러)	2	화면

```
$ ls -l > output.txt                    # 표준출력을 파일로 (1>과 동일)
$ javac Hello.java 2> error.txt         # 에러 메시지만 파일로
$ javac Hello.java > out.txt 2> error.txt   # 출력/에러 분리 저장
$ wc -l < input.txt                     # 입력 리다이렉션
$ ps -ef | grep httpd | wc -l           # 파이프: 앞 출력 → 뒤 입력
```

5-2 체크포인트

`ls /no/such/dir > error.log` 로 하면? 에러 메시지는 **stderr(2)**로 나가므로 `>` (stdout)에는 잡히지 않음 → error.log는 **빈 파일**, 에러는 화면에 출력.

기본 명령어

명령	기능
`pwd`	현재 작업 디렉터리
`whoami`	현재 사용자 이름
`who`	로그인한 모든 사용자
`which cmd`	명령어의 전체 경로
`su - 계정`	계정 전환, exit로 복귀

기호	의미
`/`	루트 디렉터리
`.`	현재 디렉터리
`..`	상위(부모) 디렉터리
`~`	홈 디렉터리
`-`	바로 이전 디렉터리 (`cd -`)

파일 다루기

명령	기능	명령	기능
`cp a b`	복사	`mkdir d`	디렉터리 생성 (`-p` : 중간 경로까지)
`mv a b`	이동/이름 변경	`rmdir d`	**비어 있는** 디렉터리만 삭제
`touch f`	빈 파일 생성	`rm -r d`	안의 내용까지 재귀 삭제
`cat`	내용 전체 출력	`head`	처음 10줄 (`-n N`)
`more`	한 페이지씩 (아래로만)	`tail -f`	끝부분 실시간 (로그 모니터링)
`less`	위아래 이동 가능 (f/b, j/k, /검색, q)		

심볼릭 링크 vs 하드 링크 ★★

	심볼릭 링크 (soft link) — `ln -s 원본 링크`	하드 링크 — `ln 원본 링크`
참조 대상	원본의 **경로(path)**를 가리키는 바로가기	원본과 **같은 데이터(inode)**를 직접 가리킴
원본 삭제 시	경로가 끊어져 **깨짐(broken link)**	하드링크 이름이 남아 있으면 **데이터 유지**
디렉터리	가능	**불가**
다른 파일시스템	가능	**같은 파일시스템 안에서만**
`ls -l` 표시	`lrwxrwxrwx ... link -> 원본경로`	일반 파일처럼 보임

5-2 실습 결과

`ln -s ~/project/config/app.conf ~/app.conf`, `ln ~/project/data/data.txt ~/data_backup.txt` 후 원본 둘 다 삭제 → `cat ~/app.conf`는 **No such file or directory**, `cat ~/data_backup.txt`는 **sample data** 그대로 출력.

권한 — ls -al · chmod · chown ★★

`drwxr-xr-x`: 첫 글자 = 종류(`d` 디렉터리, `-` 파일, `l` 링크), 다음 9글자 = User · Group · Other 각각 rwx. `ls -al`의 `-a`는 숨김 파일 포함, `-l`은 상세 정보.

권한	값	파일	디렉터리
r (read)	4	내용 보기(cat, less)	`ls`로 목록 보기
w (write)	2	내용 수정·삭제	안에 파일·폴더 생성·삭제
x (execute)	1	프로그램·스크립트 실행	`cd`·디렉터리 **통과**

디렉터리 r vs x

x 없이 r만 있으면 **목록은 보여도** 안에 들어가거나 파일을 열 수 없음.

```
rw-rw-r--  →  6 6 4
$ chmod 664 file    # 숫자 모드
$ chmod 600 file    # rw-------
$ chmod a+x file    # 모두에게 실행
$ chmod u-rx file   # 소유자 r,x 제거
$ chmod go+w file   # 그룹·기타 w 추가
```

u=user, g=group, o=other, a=all · +추가, -제거

```
$ sudo chown ec2-user f           # 소유자
$ sudo chown ec2-user:webteam f   # 소유자:그룹
$ sudo chgrp webteam f            # 그룹만
```

chmod = **권한** 변경 / chown·chgrp = **소유자·그룹** 변경(대부분 **sudo 필요**). sudo 없이 chown하면 `Operation not permitted`.

자주 쓰는 숫자	777	755	644	600	400
문자	rwxrwxrwx	rwxr-xr-x	rw-r--r--	rw-------	r--------

find · grep · xargs ★

```
$ find . -name "*.java"                    # 이름으로 검색
$ find /var/log -type f -mtime -1          # 최근 1일 이내 수정된 일반 파일
$ find . -size +100M                       # 100MB보다 큰 파일
$ find . -name "*.java" | xargs grep -i "hello"    # 찾은 파일들 안에서 검색
```

`-mtime -1`	`-mtime 1`	`-mtime +1`
0~24시간 (최근 1일 이내)	24~48시간 (딱 하루 전)	**48시간~** (그보다 오래됨)

`-type f` 일반 파일 / `-type d` 디렉터리. 경과 시간을 24로 나눠 소수점을 버린 일수로 비교하므로 `+1`은 2일 이상.

grep 옵션	기능
`-r`	하위 디렉터리까지 재귀 검색
`-n`	줄 번호 표시
`-i`	대소문자 무시
`-H`	파일 이름 항상 표시
`"t[wo]o"`	정규식: two, too 매칭

`find | xargs grep` 은 `grep -ir "hello" .` 과 비슷하지만 find로 검색 대상을 더 정교하게 지정 가능.

sed · awk

sed — Stream EDitor (찾아서 바꾸기)

```
$ sed 's/foo/bar/g' file.txt      # 화면 출력만
$ sed -i 's/foo/bar/g' file.txt   # 원본 덮어씀
$ sed -i.bak 's/a/b/' file.txt    # 백업 후 수정
$ sed -n '73,87p' file            # 73~87줄 출력
```

기본은 원본을 고치지 않고 결과만 출력. `-i` 는 원본 수정.

awk — 필드 기반 처리 (공백 기준)

```
$ awk '{print $1}' access.log      # 1번째 필드 (IP)
$ awk '{print $1, $9}' access.log  # IP와 상태코드
```

access.log 기준: `$1` IP · `$6~8` 요청 · `$9` 상태 코드 · `$NF` 마지막 필드(응답 크기)

집계 파이프라인 ★

`awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -3`

awk: IP만 추출 → sort: 같은 값끼리 붙게 정렬(**uniq는 연속된 중복만 셈**) → uniq -c: 중복 합치고 횟수 붙임 → sort -rn: 숫자(-n) 내림차순(-r) → head -3: 상위 3개

tar · 셸 스크립트 · wget/curl

```
$ tar -czvf archive.tar.gz /path/dir     # 압축: c 생성, z gzip, v 진행표시, f 파일명
$ tar -xzvf archive.tar.gz               # 해제: x extract
$ chmod a+x sample.sh && ./sample.sh     # 스크립트 실행 (첫 줄 #!/bin/bash)
$ wget https://example.com/file.png      # 다운로드 전용, 기본 동작이 저장
$ curl -O https://.../file.png           # -O 원격 파일명으로 저장
$ curl -o sample.html https://example.com # -o 지정한 이름으로 저장
$ curl -I https://example.com            # 응답 헤더만
$ curl -X POST -d '{"name":"kim"}' URL   # 서버로 데이터 전송
```

wget = 다운로드 전용 / **curl**(Client URL) = HTTP·FTP·SMTP 등 범용 통신 클라이언트, 다운로드는 기능 중 하나. AL2023에는 curl 기본 설치, wget은 `dnf install wget` 필요할 수 있음.

5-2 백업 자동화

`tar -czvf ~/project_backup_$(date +%F).tar.gz .` — `$(date +%F)` 가 오늘 날짜(YYYY-MM-DD)를 파일명에 넣음. `cat > 파일 << 'EOF' ... EOF` 로 여러 줄 파일을 한 번에 작성.

.bash_profile vs .bashrc · 환경변수 ★

.bash_profile	.bashrc
로그인할 때 한 번 실행, 로그인 세션 초기 설정	**새 셸(터미널)을 열 때마다** 실행, alias·PATH 등 개인화

로그인 시 .bash_profile → (내부의 `source ~/.bashrc`) → .bashrc 순으로 둘 다 실행. 이후 새 터미널·서브셸에서는 .bashrc만.

```
alias ll='ls -alF'
alias proj="cd ~/project"
export PATH="$HOME/myscripts:$PATH"   # export → 자식 프로세스에도 전달
$ source ~/.bashrc                     # 수정 내용을 지금 셸에 즉시 적용
$ env | grep PATH                      # 환경변수 확인
```

5-2 체크포인트

.bashrc를 고쳐도 같은 터미널에서 바로 안 되는 이유: .bashrc는 셸이 **시작될 때** 읽히기 때문 → `source ~/.bashrc` 또는 새 터미널. PATH 등록 전 `hello.sh`는 **command not found**, 등록 후에는 경로 없이 실행.

5-2 부록 — 자주 나는 오류

오류	원인	해결
Permission denied	실행 권한 없음 / sudo 필요	`chmod +x` 또는 sudo
command not found	오타 또는 미설치	철자 확인, `sudo dnf install`
No such file or directory	경로 오타, 심볼릭 링크 원본 삭제	`pwd`, `ls`로 확인
Operation not permitted (chown)	소유자 변경은 sudo 필요	sudo 붙여 재실행
alias/PATH 미적용	.bashrc 반영 안 함	`source ~/.bashrc`
Connection timed out (SSH)	보안 그룹 22번 미허용	SSH(22) 인바운드 확인
화면이 멈춘 것 같음	man, less 등 페이저 실행 중	`q`

08 Git 기초 + 6-1 로컬 저장소 실습

VCS (Version Control System)

코드의 모든 변경 이력을 **저장소(Repository)**에 기록·관리. 변경 기록 단위 = **커밋(commit)**. 원하는 시점으로 되돌릴 수 있고, **브랜치(branch)**로 기존 코드에 영향 없이 독립 개발 후 **병합(merge)**.

왜 쓰나: ① 백업 ② 이력 관리(특정 시점 복원) ③ 협업. VCS 없으면 report_final_REALLY_final.docx 지옥 / Git은 파일 1개 + 커밋 이력.

중앙집중형 vs 분산형 ★

중앙집중형 (CVS, Subversion/SVN)	분산형 (Git, Mercurial) — 사실상 표준
하나의 중앙 서버가 모든 버전 정보 보관 클라이언트는 필요할 때 checkout **서버가 다운되면 아무도 협업 불가**	각 사용자가 **전체 이력이 담긴 저장소를 통째로 복사** 원격 서버가 멈춰도 로컬에서 커밋·조회 가능 이후 원격과 동기화

Git 소개와 저장 방식

- **2005년 리누스 토르발스**가 리눅스 커널 개발 지원용으로 설계. "the stupid content tracker". 공식: git-scm.com
- GitHub, GitLab 등 대부분 호스팅 플랫폼이 Git 기반

Git — Snapshot 방식	CVS · SVN — Delta 방식
매 커밋마다 전체 상태의 **스냅샷** 저장. 바뀐 파일만 새로 저장하고 나머지는 이전 스냅샷에 대한 **링크**	"기준본 + 변경분(delta)"의 연속. 버전을 보려면 델타를 순서대로 적용, 병합이 상대적으로 느림

설치와 설정

```
$ sudo dnf install -y git                    # AL2023/RHEL (Ubuntu: apt-get, macOS: brew)
$ git --version
$ git config --global user.name "Hong Gildong"
$ git config --global user.email "gildong@example.com"
$ git config --global init.defaultBranch main # 기본 브랜치 main
$ git config --list                          # 설정 확인
```

설정은 **<홈>/.gitconfig**에 저장. 특정 저장소만 다르게 하려면 그 폴더에서 `--global` 없이. GitHub은 **2020년**부터 기본 브랜치 **master → main**. Windows는 Git Bash 또는 WSL2.

4단계 공간 ★★

Working Directory		Staging Area		Local Repository		Remote Repository
파일 수정	`git add` →	커밋할 변경을 골라 담는 임시 공간	`git commit` →	영구 기록(.git)	`git push` →	GitHub 등

스테이징 영역이 있어 원하는 파일만 골라 담아 의미 있는 단위로 커밋을 나눌 수 있음.

저장소 시작 · 기본 순환

```
$ git init                    # ① 새 프로젝트: .git 숨김 폴더 생성 (지우면 이력 전부 사라짐)
$ git clone <remote-url>      # ② 기존 프로젝트 복제
$ vi Hello.java               # MODIFY
$ git add Hello.java          # STAGE  (git add .  /  git add *.java)
$ git commit -m "메시지"      # COMMIT (-m 생략 시 편집기가 열림)
```

파일 상태 생명주기 ★

Untracked(새 파일, 추적 안 함) —add→ **Staged**(커밋 대기) —commit→ **Committed** —수정→ **Modified** —add→ Staged ...

명령	역할
`git status`	untracked / staged / modified 상태 확인 (**어떤 파일**이 바뀌었는지)
`git add <file>`	untracked·modified → staged
`git commit`	staged → committed
`git restore --staged <file>`	**스테이징만 취소** (작업 내용은 그대로)

status 메시지	의미
Untracked files	Git이 아직 추적하지 않는 새 파일
Changes to be committed	스테이징됨 (add 완료, 커밋 전)
Changes not staged for commit	추적 중인 파일이 수정됨, 아직 add 안 함
nothing to commit, working tree clean	마지막 커밋과 같음 (정상)

git log · git diff ★★

각 커밋은 고유 **해시값**으로 식별: SHA-1이면 **40자리**, SHA-256이면 64자리. 보통 **앞 7자리** 단축형 사용. 해시는 Git이 내용(스냅샷·작성자·시각·메시지·이전 커밋)으로 **자동 계산** — 커밋 메시지와 다름.

```
$ git log                          # 해시, Author, Date, 메시지
$ git log --oneline                # 한 줄 요약 (단축 해시 + 메시지)
$ git log --oneline --graph --all # 여러 브랜치를 그래프로
```

명령	비교 대상	보여주는 것
`git diff`	작업 디렉터리 ↔ **스테이징 영역**	아직 add 안 한 변경 (unstaged)
`git diff --cached`	**스테이징 영역** ↔ HEAD	커밋 예정인 변경 (staged)
`git diff HEAD`	작업 디렉터리 ↔ **HEAD(마지막 커밋)**	커밋 이후 모든 변경

> **diff 함정**
> 파일을 수정하고 `git add` 까지 한 뒤 `git diff` 를 치면 **아무것도 안 나옴** → 이때는 `git diff --cached` . 출력의 `-` 는 삭제된 줄, `+` 는 추가된 줄.

커밋 메시지: "fix bug"처럼 모호하게 X → "무엇을 왜 바꿨는지" 짧고 명확하게.

.gitignore

```
*.class      # Java 클래스 파일
*.exe
*.[oa]       # .o 또는 .a
temp/        # 디렉터리 전체 (끝에 /)
```

프로젝트 루트에 작성하면 대상 파일이 `git status` 의 Untracked 목록에서 사라짐. 생성 도구: gitignore.io(toptal), github/gitignore 템플릿, IDE 자동 생성.

6-1 실습 흐름 (~/hello-git)

1. Part 1: `git config` → `git init` → `cat > Hello.java << 'EOF'` 로 파일 작성 → `javac Hello.java` (Hello.class 생성, **바이트코드**) → `java Hello` 실행
2. `git status` : Hello.class, Hello.java, README.md 모두 Untracked
3. `git add Hello.java README.md` 만 → Hello.class는 의도적으로 제외 → 첫 커밋 "Initial commit..."
4. Part 2: `sed -i 's/Hello, world from GIT!/Hi, Git!/' Hello.java` → status는 **modified**, diff는 **- / + 줄** → 두 번째 커밋 "Update greeting message"
5. `mkdir temp; touch temp/cache.tmp` → `.gitignore` 에 `*.class`, `temp/` → status에 `.gitignore` 만 남음 → "Add .gitignore" 커밋 (총 3개)

6-1 자주 만나는 오류	원인 · 해결
javac: command not found	JDK 미설치 → `sudo dnf install -y java-17-amazon-corretto-devel`
Please tell me who you are	user.name / user.email 미설정 → `git config`
fatal: not a git repository	저장소 폴더 밖에서 실행 → `cd ~/hello-git`
nothing to commit, working tree clean	변경사항 없음 (정상)

09 Git 원격 저장소 + 7-1 실습

브랜치와 커밋 복습

커밋 = 특정 시점의 스냅샷 / **브랜치** = 커밋이 이어진 독립적인 작업 흐름 (예: main = 배포 버전, dev = 새 기능, hotfix = 긴급 수정).

로컬 vs 원격 저장소

로컬 = 내 컴퓨터(EC2)의 `.git` / 원격 = 인터넷에 호스팅된 Git 저장소(GitHub, GitLab, Bitbucket). 로컬 → 원격: **push** · 원격 → 로컬: **clone / pull**.

GitHub 계정 · 저장소 만들기

- Sign up → username 확정(주소: `github.com/<username>/<repo>`) → **2FA** 설정 (GitHub가 단계적으로 의무화)
- + → New repository. **로컬에 이미 저장소가 있으면 README·.gitignore·license 없이 완전히 빈 저장소로** 생성 (충돌 방지)
- 2020년 10월부터 GitHub 기본 브랜치명은 **main**

git remote ★

```
$ git remote add origin https://github.com/kim/hello-git.git   # git remote add <이름> <URL>
$ git remote -v                          # 목록 (fetch / push)
$ git remote rename origin upstream      # 이름 변경
$ git remote remove upstream             # 연결 해제
```

origin은 관례적인 기본 별칭(필수 아님). remote 이름은 **내 .git 설정에만** 저장되고 push해도 원격에 올라가지 않음.

인증 — PAT (Personal Access Token) ★

계정 비밀번호	PAT
2021년 8월부터 git push/pull 인증에 사용 불가 유출 시 계정 전체 위험, 범위·기한 제한 불가 웹사이트 로그인에만 사용	비밀번호 자리에 입력하는 긴 문자열(**ghp_**로 시작) 필요한 권한(scope)만 골라 발급, 만료 기한 지정 유출돼도 토큰만 삭제하면 계정 안전

- 발급: Settings → Developer settings → Personal access tokens → **Tokens (classic)** → Note · Expiration(7 days 등 짧게) · Scopes
- `public_repo`만 체크 → 공개 저장소에 push 가능, Private 접근 불가 / Private이면 `repo` 전체
- 토큰은 생성 화면을 벗어나면 **다시 볼 수 없음**. 비밀번호와 같으므로 공유 금지
- **Fine-grained 토큰은 Collaborator로 초대받은 저장소에 push 불가** → 실습은 classic 사용

```
$ git config --global credential.helper store   # 토큰 저장 (~/.git-credentials, 평문)
```

store는 평문 저장 → 본인 전용 PC·서버에서만. 암호화 저장소: Windows GCM, macOS osxkeychain, Linux libsecret. **7-1: push 전에 설정해야** 첫 push에서 입력한 토큰이 저장됨.

push · -u · clone · pull ★

```
$ git push origin main          # git push <원격> <브랜치>
$ git push -u origin main       # -u(--set-upstream): 로컬 main ↔ origin/main 연결 (처음 한 번)
$ git push                      # 이후 이름 생략 가능
$ git pull                      # 원격의 최신 변경 받기
$ git clone https://github.com/kim/hello-git.git hello-git-cloned   # 새 폴더 이름 지정
```

- **clone**은 원격의 커밋 이력 전체를 복제하고 **origin이 자동 연결**됨. upstream도 설정돼 바로 `git push` 가능
- clone 시 지정한 폴더 이름(hello-git-cloned)은 내 컴퓨터 폴더명일 뿐, remote 이름(origin)과 무관

태그 (Tag) ★★

특정 커밋에 붙이는 **이름표**, 주로 릴리스 버전 표시. 브랜치와 달리 **위치가 고정**.

```
$ git tag -a v1.0.0 -m "First stable release"       # 최신 커밋에 Annotated Tag
$ git tag -a v0.9.0 a1b2c3d -m "Beta release"       # 과거 커밋 (해시 지정)
$ git tag                                           # 목록
$ git show v1.0.0                                   # 상세 (Tagger, Date, 메시지, 커밋)
$ git push origin v1.0.0                            # 태그 하나 push
$ git push origin --tags                            # 모든 태그 push
```

태그 함정

`-a` (Annotated) 태그에는 **작성자·날짜·메시지**가 함께 저장. **`git push` 만으로는 태그가 원격에 올라가지 않음** → 따로 push. GitHub **Releases**에서 태그로 릴리스 노트 작성. 과거 커밋의 해시는 `git log --oneline` 에서 확인.

7-1 협업 실습 흐름 (Alice = 저장소 주인, Bob = 초대받는 사람)

단계	내용
Part 1 (개인)	빈 저장소 생성 → classic PAT 발급 → `git remote add origin` → `credential.helper store` → `git push -u origin main` → GitHub에서 확인 (**Hello.class·temp/는 없어야 정상**, .gitignore가 원격에도 적용) → README 수정 후 `git push` 는 토큰 안 물어봄
[Alice] 초대	Settings → Collaborators → Add people. Bob은 메일·알림에서 수락. **수락 안 해도 Public이라 clone(읽기)은 되지만 push(쓰기)는 거부**
[Bob] clone	`git clone ... hello-git-cloned` . Public이라 인증 불필요. Hello.class가 없으니 직접 `javac`
[Bob] 수정 → push	sed로 수정 → diff → add → commit → `git push` (Bob 본인의 PAT로 인증)
[Alice] pull	`git pull` → **Fast-forward**, 맨 위에 Bob의 커밋
[Alice] 태그	`git tag -a v1.0.0` → `git push origin v1.0.0` . Bob은 `git pull` 후 `git tag` 로 확인
정리	공용 EC2면 `rm ~/.git-credentials` , 안 쓰는 PAT 삭제

10 헷갈리는 개념 비교 모음

비교	A	B
Free Software vs OSS	FSF, **윤리적**, 4대 자유	OSI, **실용적**, 협업·검증으로 품질
패키지형 vs 프로젝트형 OSS	부품 단위(라이브러리)	완제품 단위(OS 전체)
저작권 vs 특허	창작과 동시에 **자동 발생**	**별도 출원**해 인정
Patent Grant vs Retaliation	기여자의 특허를 **무상 허락**	특허 소송 걸면 라이선스 **자동 종료**
LGPL 동적 vs 정적 링크	동적: 응용 소스 공개 의무 없음	정적: 재링크 가능하게 오브젝트 파일 등 제공
LGPL vs MPL 공개 범위	LGPL: 라이브러리 수정분 (링크된 코드 제외)	MPL: **파일 단위**
GPLv2 vs GPLv3	v2: 특허 조항 없음, Apache 2.0과 비양립	v3: Tivoization 금지, 특허 조항, Apache 2.0과 양립
On-Premise vs Cloud	직접 구축, 고정 비용, 직접 운영	즉시 사용, 종량제, 사업자 운영
루트 vs IAM 사용자	모든 권한, 결제 변경 등에만	평소 작업용, AdministratorAccess
Region vs AZ	지리적 데이터센터 클러스터	Region 안의 물리적으로 분리된 데이터센터
EC2 Stop vs Terminate	EBS 비용 계속, 재시작 가능	완전 삭제, 되돌릴 수 없음
Private Key vs Public Key	사용자 보관(.pem), R을 **암호화**	EC2 보관(~/.ssh), **복호화**해 확인
RPM vs DNF	저수준, 의존성 수동	저장소 사용, 의존성 자동
dnf search vs list	이름+요약에서 키워드	이름(패턴)만
rpm vs deb	Red Hat 계열, dnf/yum	Debian 계열, apt/apt-get
`>` vs `2>` vs `>>`	`>` stdout 덮어쓰기, `2>` stderr	`>>` 이어 붙이기 (7-1 README 예시)
심볼릭 vs 하드 링크	경로 참조, 원본 삭제 시 깨짐, 디렉터리·다른 FS 가능	inode 참조, 원본 삭제해도 유지, 디렉터리·다른 FS 불가
chmod vs chown	권한 변경 (내 파일이면 sudo 불필요)	소유자 변경 (대부분 sudo)
more vs less	아래로만	위아래 이동
wget vs curl	다운로드 전용	범용 통신 (-O, -o, -I, -X POST)
curl -O vs -o	원격 파일명 그대로 저장	지정한 이름으로 저장
sed vs sed -i	화면 출력만	원본 파일 수정
.bash_profile vs .bashrc	로그인 시 1회	새 셸마다
중앙집중형 vs 분산형 VCS	CVS·SVN, 서버 다운 시 협업 불가	Git·Mercurial, 전체 사본 보유
Snapshot vs Delta	Git: 커밋마다 전체 스냅샷(변경 없는 파일은 링크)	SVN: 기준본 + 변경분
커밋 메시지 vs 해시	사용자가 작성, 중복 가능	Git이 자동 계산, 고유
git status vs git diff	**어떤 파일**이 바뀌었는지	**어떤 줄**이 바뀌었는지
git diff vs --cached	작업 디렉터리 ↔ 스테이징	스테이징 ↔ HEAD
git init vs clone	새 저장소 생성	기존 원격 저장소 복제 (origin 자동)
push vs pull	로컬 → 원격	원격 → 로컬
브랜치 vs 태그	커밋이 쌓이며 이동하는 작업 흐름	특정 커밋에 고정된 이름표
classic vs fine-grained PAT	Collaborator 저장소 push 가능	Collaborator 저장소 push 불가

11 연도 · 숫자 암기표

연도

연도	사건
1983	GNU 프로젝트
1985	FSF 설립
1989	GPL v1
1991	Linux 커널 공개
1998	OSI 설립
1999	Apache 재단
2005	Git 공개
2006	AWS 클라우드 서비스 시작
2007	GPL v3
2008	GitHub 창립
2012	MPL 2.0 (GPL 결합 허용)
2014	Docker 공개
2015~	TensorFlow, PyTorch
2020(10월)	GitHub 기본 브랜치 main
2021년 8월	GitHub 비밀번호 인증 중단 → PAT
2023~	오픈 웨이트 LLM 경쟁
2024년 6월	CentOS 7 EOL (전통 CentOS 종료)

숫자

항목	값
FSF 자유	4대 자유
OSI 기준 / 강의 핵심 원칙	10개 / 5개
OSI 인증 라이선스	약 80개
전체 OSS 라이선스	약 2,000여 종
Permissive / MIT 포함 / Copyleft	78% / 92% / 22%
라이선스 충돌 발견율(2026)	68%
클라우드 점유율 AWS/Azure/GCP	28% / 21% / 14% (합 63%)
Free Plan 크레딧	$100 + 최대 $100, 최대 6개월
카드 인증 결제 대기	1 USD(100원), 3~5일
서울 리전 AZ 수	4개 (2a~2d)
SSH / HTTP 포트	22 / 80 (TCP)
t3.micro	vCPU 2, RAM 1GiB, $0.013/시간
stdin / stdout / stderr	0 / 1 / 2
r / w / x	4 / 2 / 1
head 기본 줄 수	10줄
커밋 해시 (SHA-1 / SHA-256)	40자리 / 64자리, 단축 7자리
Git 공간	4단계
vi 모드	3가지
SW 인력 증가율 / 충원 계획 기업 / 경력:신입	0.2% / 24% / 2.8배

12 예상 문제

정답·해설은 다음 페이지

A. 객관식 · O/X

1. 무료로 배포되지만 소스가 공개되지 않은 메신저 프로그램은 OSS인가? (O/X)

2. OSI 핵심 원칙 중 "특정 개인·단체·사용 분야·기술을 차별해서는 안 된다"는 원칙의 이름은?

3. 다음 중 독점(폐쇄형) SW와 결합할 수 **없는** 라이선스는? ① LGPL ② MPL ③ GPL ④ Apache ⑤ BSD

4. GPL v3에서 v2 대비 추가된 것이 **아닌** 것은? ① Tivoization 금지 ② 특허 조항 ③ Apache 2.0과의 호환성 ④ 동적 링크 시 소스 공개 면제

5. GPLv3 소스를 수정한 프로그램을 회사 내부에서만 쓰면 소스를 공개해야 한다. (O/X)

6. MPL 1.1 코드와 GPLv3 코드를 결합한 프로그램을 배포할 수 없는 이유를 쓰시오.

7. Linux 커널이 여전히 사용하는 GPL 버전과 그 이유는?

8. 클라우드 컴퓨팅의 세 가지 특징(On-Demand 외 2개)을 쓰시오.

9. AWS에서 루트 계정 대신 평소에 사용하도록 권장되는 계정과, 루트 계정에 반드시 설정해야 하는 보안 기능은?

10. EC2 인스턴스를 중지(Stop)하면 비용이 전혀 발생하지 않는다. (O/X)

11. Key Pair 인증에서 EC2는 무엇으로 [R]Alice를 복호화하며, 언제 인증에 성공하는가?

12. 서울 리전 이름과 AZ 개수는? 4대의 서버를 배치할 때 실무에서 가장 흔한 절충안은?

B. 리눅스 명령어

13. vi에서 파일 전체의 apple을 banana로 모두 바꾸는 명령은?

14. `httpd-2.4.68-1.amzn2023.0.1.x86_64` 에서 버전과 릴리스는?

15. RPM의 단점과 이를 해결한 도구는? 그 도구의 저장소 설정 경로는?

16. `javac Hello.java > out.txt 2> err.txt` 에서 컴파일 에러 메시지는 어디에 저장되는가?

17. `rw-r-----` 를 숫자로 쓰면? `chmod 751` 을 문자로 쓰면?

18. 원본 파일을 삭제한 뒤에도 내용을 읽을 수 있는 링크는 무엇이며, 그 이유는?

19. `find /var/log -type f -mtime +1` 이 찾는 파일은?

20. `awk '{print $9}' access.log | sort | uniq -c` 에서 sort를 빼면 어떤 문제가 생기는가?

21. `sed 's/foo/bar/g' a.txt` 실행 후 a.txt의 내용은 바뀌는가? 원본을 바꾸려면?

22. tar로 `hw1` 폴더를 `hw1.tar.gz` 로 압축하는 명령과 해제 명령을 쓰시오.

23. .bashrc에 alias를 추가했는데 같은 터미널에서 적용되지 않는다. 이유와 해결 방법은?

24. sudo 없이 `chown root:root notes.txt` 를 실행하면 어떻게 되는가?

C. Git

25. 중앙집중형 VCS의 가장 큰 위험과, 분산형 VCS가 이를 극복하는 방식은?

26. Git의 Snapshot 방식과 SVN의 Delta 방식의 차이를 쓰시오.

27. 파일을 수정하고 `git add` 까지 했다. 이 변경을 보려면 어떤 명령을 써야 하는가?

28. 실수로 스테이징한 파일을, 작업 내용은 유지한 채 스테이징만 취소하는 명령은?

29. .gitignore에 `*.class` 를 적기 전과 후 `git status` 는 어떻게 달라지는가?

30. 모든 학생이 같은 명령을 실행해도 커밋 해시가 다른 이유는?

31. 2021년 8월 이후 GitHub에 push할 때 비밀번호 자리에 입력하는 것은? 장점 2가지는?

32. `git push -u origin main`에서 `-u`의 역할은?

33. `git clone`으로 받은 저장소의 원격 이름은? 폴더 이름을 바꿔 clone하면 원격 이름도 바뀌는가?

34. 과거 커밋 a1b2c3d에 주석 태그 v0.9.0을 달고 원격에 올리는 명령 2줄을 쓰시오.

35. 로컬 저장소가 이미 있을 때 GitHub에 저장소를 만들면서 README를 추가하지 말아야 하는 이유는?

36. Collaborator 초대를 수락하지 않은 Bob이 Public 저장소에 할 수 있는 것과 없는 것은?

12 정답 · 해설

1. **X** — 소스 공개·협업이 안 되므로 OSS가 아님 (예: 카카오톡).
2. **무차별 원칙**.
3. ③ **GPL** — 2차 저작물 공개 의무가 있어 독점 SW와 결합 불가.
4. ④ — 동적 링크 시 응용 소스 공개 면제는 LGPL의 특징.
5. **X** — 공개 의무는 **배포**할 때 생김. 내부 사용만 하면 의무 없음.
6. GPL은 결합 전체를 GPL로 공개하라고 요구하지만, MPL 코드는 MPL로만 배포하도록 허락되어 GPL로 재배포할 권한이 없음 → 어느 쪽을 지켜도 다른 쪽을 어김 → 양립 불가.
7. **GPLv2** — 토발즈가 v3의 Tivoization 금지 조항에 반대.
8. **Pay-as-you-go**(쓴 만큼 지불), **Self-Service**(콘솔에서 직접 구성).
9. **IAM 관리자 사용자**(AdministratorAccess), 루트에는 **MFA**.
10. **X** — 컴퓨팅 비용은 멈추지만 **EBS 스토리지 비용**은 계속. 완전 중단은 Terminate.
11. Alice의 **공개키**로 복호화해 얻은 R′이 보낸 난수 R과 **같으면** 성공.
12. `ap-northeast-2`, **4개**. **2개 AZ에 2대씩**.
13. `:%s/apple/banana/g`
14. 버전 **2.4.68**, 릴리스 **1.amzn2023.0.1** (아키텍처 x86_64).
15. 의존성을 사용자가 직접 해결해야 함 → **YUM/DNF**가 자동 해결. `/etc/yum.repos.d/`.
16. **err.txt** — 에러는 stderr(2)로 출력됨.
17. **640** / **rwxr-x--x**.
18. **하드 링크** — 경로가 아니라 같은 데이터(inode)를 직접 가리키므로 이름 하나가 남아 있으면 데이터 유지.
19. /var/log 아래에서 수정된 지 **48시간(2일) 이상** 지난 일반 파일.
20. uniq는 **연속된** 중복만 합치므로, 같은 값이 떨어져 있으면 따로 세어져 집계가 틀림.
21. 바뀌지 **않음**(화면 출력만). `sed -i 's/foo/bar/g' a.txt` (백업은 `-i.bak`).
22. `tar -czvf hw1.tar.gz hw1` / `tar -xzvf hw1.tar.gz`
23. .bashrc는 셸이 **시작될 때** 읽히기 때문. `source ~/.bashrc` 또는 새 터미널.
24. **Operation not permitted** — 소유자 변경은 관리자 권한(sudo) 필요.
25. 중앙 서버가 다운되면 아무도 협업 불가 → 분산형은 각자 **전체 이력 사본**을 가져 로컬에서 커밋·조회 계속, 나중에 동기화.
26. Git은 커밋마다 전체 **스냅샷**(바뀌지 않은 파일은 이전 스냅샷 링크), SVN은 기준본 + **변경분(delta)**을 순서대로 적용.
27. `git diff --cached` (그냥 `git diff`는 작업 디렉터리 ↔ 스테이징이라 아무것도 안 나옴).
28. `git restore --staged <file>`
29. 전: Untracked files에 .class 파일이 표시 → 후: 목록에서 사라짐 (.gitignore 자신만 Untracked로 남음).
30. 해시는 파일 내용뿐 아니라 **작성자 이름·이메일, 커밋 시각, 메시지, 이전 커밋 해시**까지 포함해 계산되기 때문.
31. **PAT**. 필요한 권한(scope)만 발급 · 만료 기한 지정 · 유출 시 토큰만 삭제하면 계정 안전 (중 2개).
32. 로컬 main과 원격 origin/main을 연결(upstream 설정) → 이후 `git push`, `git pull`만으로 동기화.
33. **origin**. 바뀌지 **않음** — 폴더 이름과 remote 이름은 별개 (remote 이름은 `git remote rename`으로 변경).
34. `git tag -a v0.9.0 a1b2c3d -m "Beta release"`
    `git push origin v0.9.0` (모두: `git push origin --tags`)
35. 원격에 로컬과 다른 커밋이 생겨 첫 push 때 **충돌**이 나기 때문 → 완전히 빈 저장소로 생성.
36. clone(읽기)은 가능, **push(쓰기)는 거부**.

> **마지막 점검**
> 시험 전날에는 10장(비교 모음)과 11장(암기표)을 먼저 보고, 틀린 문제의 장으로 돌아가 해당 표만 다시 확인하세요.